이미지 기반 알약 객체 검출 및 분류를 위한 2-Stage 헬스케어 AI 시스템 개발

3팀
김경제, 김태민, 신기성, 채영환

초록(Abstract)

본 프로젝트는 사용자가 촬영한 알약 이미지를 기반으로, 사진 속 복수의 알약 위치를 검출하고 각 알약의 종류를 식별하는 2-stage 헬스케어 AI 시스템을 개발하는 것을 목표로 하였다. 이를 위해 Stage 1에서는 객체 검출 모델을 활용하여 이미지 내 알약의 위치를 탐지하고, Stage 2에서는 검출된 개별 알약 이미지를 분류 모델에 입력하여 세부 클래스를 예측하는 구조를 설계하였다. 초기 단계에서는 SSD, YOLOv8, YOLOv11+ResNet 계열의 baseline 성능을 비교하였으며, 그 결과 YOLOv11s와 ResNet50을 결합한 2-stage 구조가 가장 우수한 성능을 보였다. 이후 데이터셋의 품질과 클래스 분포를 지속적으로 분석하며 V1부터 V6에 이르는 데이터셋 개선 과정을 수행하였고, 특히 누락 클래스 보강, 앞·뒷면 데이터 불균형 해결, Grayscale 및 Sharpen 기반 증강 실험, 저빈도 클래스 데이터 추가 확보, 클래스별 맞춤형 증강 설계 등을 통해 모델 성능을 향상시켰다. 최종적으로 4,719장의 학습 이미지를 바탕으로 YOLOv11s + ResNet50 구조에 적절한 augmentation을 적용한 실험에서 Public Score(mAP) 0.98569를 달성하였다. 또한 본 보고서는 최종 모델이 높은 성능을 보였음에도 불구하고 closed-set classification 구조에 따른 한계를 가진다는 점을 분석하고, 향후 unknown 클래스 탐지 및 신규 클래스 확장성을 확보하기 위한 embedding head 기반 분류 구조를 개선 방향으로 제안한다.

목차

1. 서론

2. 프로젝트 개요 및 연구 목표

3. 팀 구성 및 역할 분담

4. 협업 방식 및 프로젝트 운영 전략

5. 데이터셋 구축 및 변화 과정

6. Baseline 모델 구축 및 비교

7. 데이터셋 및 증강 기반 성능 개선 실험

8. 최종 모델 선정 및 성능 분석

9. 모델의 한계점

10. 향후 개선 방향

11. 결론

1. 서론

최근 헬스케어 분야에서는 사용자의 일상 데이터를 인공지능 기반으로 분석하여 실질적인 건강 정보를 제공하는 서비스가 빠르게 확산되고 있다. 그중에서도 복약 관리와 관련된 서비스는 개인 건강과 직접적으로 연관되어 있어 높은 활용 가치를 가진다. 특히 사용자가 자신이 복용 중인 약의 이름이나 성분을 정확히 알지 못하는 경우가 많고, 다수의 약을 동시에 복용하는 상황에서는 오복용이나 약물 상호작용에 대한 위험도 증가할 수 있다. 이러한 문제를 해결하기 위해서는 사용자가 촬영한 알약 사진만으로 약의 종류를 식별하고, 해당 결과를 기반으로 후속 정보를 제공할 수 있는 기술이 필요하다. 본 프로젝트는 이러한 실용적 요구를 배경으로 수행되었으며, 이미지 인식 기술을 헬스케어 서비스에 접목하는 실제적인 응용 사례를 목표로 하였다.

알약 인식 문제는 일반적인 이미지 분류 문제와는 다른 복합적인 난점을 가진다. 하나의 이미지 안에 여러 개의 객체가 동시에 존재할 수 있으며, 각 객체는 크기가 작고 외형이 매우 유사하다. 더욱이 많은 알약은 모양과 색상만으로는 구분이 어렵고, 표면에 각인된 식별 문자나 앞·뒷면의 세부 차이가 분류의 핵심 단서가 된다. 그러나 실제 촬영 환경에서는 조명, 회전, 해상도, 문자 가독성, 배경 차이 등의 요소로 인해 이러한 구분 단서가 손쉽게 손실된다. 따라서 본 문제는 단순한 객체 검출이나 단순 분류만으로 해결되기 어렵고, 검출과 분류를 함께 고려한 정교한 시스템 설계가 요구된다.

본 프로젝트에서는 이러한 문제를 해결하기 위해 2-stage 구조를 설계하였다. Stage 1은 이미지 속 알약의 위치를 검출하는 역할을 수행하며, Stage 2는 검출된 개별 알약 crop을 기반으로 세부 클래스를 분류하는 역할을 수행한다. 이 구조는 위치 검출과 fine-grained classification을 분리하여 각 단계가 보다 명확한 역할에 집중할 수 있도록 한다. 발표자료의 baseline 비교 결과에서도 SSD, YOLOv8, YOLOv11+ResNet 구조를 비교한 끝에, 검출과 분류를 역할 분리한 YOLOv11s + ResNet50 구조가 가장 우수한 성능을 보였음이 확인되었다.

2. 프로젝트 개요 및 연구 목표

본 프로젝트의 핵심 목표는 사용자가 모바일 환경에서 촬영한 알약 사진으로부터, 이미지 속 알약의 위치와 종류를 자동으로 인식하는 시스템을 개발하는 것이었다. 발표자료에서는 이를 세 가지 키워드로 정리하고 있다. 첫째는 **Object Detection**으로, 사진 속에서 각 알약의 위치를 bounding box 형태로 검출하는 작업이다. 둘째는 **Classification**으로, 검출된 개별 알약이 어떤 클래스에 해당하는지를 식별하는 작업이다. 셋째는 **App Support** 관점으로, 모델의 최종 결과가 사용자에게 실제 정보를 제공할 수 있는 서비스 구조로 연결되는 것이다. 이러한 세 요소는 단순 모델 구현을 넘어, 실제 헬스케어 스타트업 서비스의 프로토타입을 구성한다는 점에서 중요한 의미를 가진다.

연구 목표는 단순히 높은 점수 하나를 달성하는 데 있지 않았다. 본 프로젝트는 실제로 알약 인식이 실패하는 원인을 구체적으로 분석하고, 그 원인에 따라 데이터셋과 모델을 단계적으로 수정하며 성능을 향상시키는 과정을 중시하였다. 예를 들어, 일부 클래스는 학습 데이터 자체가 부족하였고, 일부 클래스는 앞면과 뒷면의 식별 문자가 상이하였으며, 또 다른 클래스는 색상과 형태는 유사하지만 문자 정보가 분류의 핵심 역할을 하는 경우였다. 이와 같이 서로 다른 오류 유형이 존재하였기 때문에, 모든 문제를 하나의

augmentation이나 단일 모델 변경으로 해결할 수는 없었다. 따라서 본 프로젝트는 실험 결과를 해석하고, 문제 클래스별 원인을 구분하며, 데이터 보강과 증강 설계를 포함한 반복적 개선 전략을 채택하였다.

3. 팀 구성 및 역할 분담

본 프로젝트는 팀 단위 협업을 기반으로 수행되었으며, 각 구성원은 서로 다른 역할을 담당하면서 전체 시스템 완성도를 높이는 방향으로 기여하였다. 발표자료의 팀 소개 페이지에 따르면, 팀은 프로젝트 매니저, 데이터 엔지니어, 모델 아키텍트, 실험 리드의 구조로 역할이 구분되어 있었다. 이러한 역할 구분은 프로젝트 초기 단계에서부터 이루어졌으며, 단순한 명목상의 역할이 아니라 실제 데이터 정제, 실험 설계, 모델 비교, 일정 관리와 연결되어 운영되었다.

프로젝트 매니저는 전체 일정과 업무 흐름을 조율하고, 프로젝트의 방향성과 우선순위를 관리하는 역할을 수행하였다. 데이터 엔지니어는 원천 데이터와 라벨 데이터를 검토하고, stage1과 stage2에 필요한 형태로 데이터를 정제하고 재구성하는 작업에 집중하였다. 모델 아키텍트는 SSD, YOLOv8, YOLOv11+ResNet 등 여러 구조를 비교하고 최종적인 2-stage 시스템 설계를 구체화하는 역할을 맡았다. 실험 리드는 augmentation 조합, class imbalance 보정, 데이터 보강 전략, 성능 비교 실험 등 반복적인 실험 설계를 주도하며 모델 개선의 방향을 설정하였다. 발표자료의 v3 stage2 데이터 가공 작업 페이지에서도 단일 객체 데이터 추출, 분류, 라벨링과 같은 작업이 역할 분담을 통해 수행되었음을 확인할 수 있다.

이와 같은 역할 분담은 프로젝트 전반의 효율성을 높이는 데 기여하였다. 특히 알약 인식 문제는 단순 모델 학습보다 데이터 품질과 클래스 분포가 성능에 큰 영향을 주는 문제이기 때문에, 데이터셋 구축과 모델 실험이 동시에 진행될 필요가 있었다. 한 명이 모든 단계를 담당하기보다, 데이터 처리와 실험 설계를 병렬적으로 수행할 수 있는 구조를 마련함으로써, 모델의 실패 사례를 빠르게 데이터 분석으로 연결하고, 그 결과를 다시 후속 실험으로 반영할 수 있었다. 이러한 운영 방식은 최종적으로 baseline 선정과 후속 성능 개선 실험의 속도와 정교함을 높이는 데 중요한 기반이 되었다.

4. 협업 방식 및 프로젝트 운영 전략

본 프로젝트는 일정표 기반의 체계적 운영과 Agile한 커뮤니케이션 방식을 결합하여 수행되었다. 발표자료의 협업 과정 페이지에서는 개발환경 세팅, 요구사항 분석, EDA, 피드백 및 수정, 모델 설계, 베이스라인 구축, 성능 개선, 발표 준비에 이르기까지 프로젝트의 모든 주요 단계가 일정표에 따라 관리되었다고 제시한다. 이는 프로젝트가 단순히 실험을 많이 수행하는 방식이 아니라, 각 단계의 목표와 산출물을 명확히 정의하고 순차적으로 진행되었음을 보여준다.

동시에, 팀은 매일 디스코드를 활용한 상시 커뮤니케이션 체계를 유지하였다. 발표자료에는 “Agile 방식으로 매일 디스코드를 활용한 지속적인 커뮤니케이션을 진행하였으며, 빠른 피드백과 즉각적인 개선이 가능했다”고 정리되어 있다. AI 프로젝트는 동일한 모델이라도 데이터셋 변경, augmentation 조정, class mapping 수정 등으로 결과가 크게 달라질 수 있기 때문에, 실험과 피드백 사이의 간격이 짧을수록 유리하다. 본 팀은 이러한 특성을 반영하여 일일 단위로 결과를 공유하고, 오분류 사례를 즉시 논의하며, 실험 계획을 빠르게 수정하는 방식으로 프로젝트를 운영하였다.

이러한 운영 전략은 특히 후반부의 V5 실험 과정에서 큰 효과를 발휘하였다. 누락 클래스 보강, 앞·뒷면 데이터 추가, Grayscale 및 Sharpen 증강 실험, issue class 데이터 추가, 클래스별 augmentation 적용과 같은 세부 실험은 모두 이전 결과의 해석과 즉각적인 의사결정을 요구하였다. 만약 이러한 과정이 주 단위 회의 중심으로 진행되었다면, 실험 간 피드백 주기가 너무 길어져 프로젝트의 반복 개선 속도가 크게 늦어졌을 가능성이 높다. 따라서 본 프로젝트에서의 협업 방식은 단순한 회의 운영이 아니라, 모델 개선 자체를 가능하게 한 핵심 기반이었다고 평가할 수 있다.

[일정 조율] 2026/03/16_healthcare_object_detection_WBS.xlsx

5. 데이터셋 구축 및 변화 과정

본 프로젝트는 하나의 고정된 데이터셋에 의존하지 않고, 문제 분석과 성능 결과에 따라 데이터셋을 V1부터 V6까지 단계적으로 발전시키는 전략을 채택하였다. 발표자료의 데이터셋 변화 페이지에 따르면, V1과 V2는 초기 정제 단계, V3는 단일 객체 데이터 보강 단계, V4는 검증용 데이터셋 단계, V5와 V6는 AI Hub 기반 확장 단계로 정리할 수 있다. 이는 데이터셋 구축이 단순한 수집 과정이 아니라, 모델 성능과 직접 연결되는 설계 요소로 기능했음을 보여준다.

5.1 V1과 V2: 초기 baseline용 데이터

V1은 Kaggle에서 제공된 원본 train 이미지 데이터를 그대로 활용한 초기 실험용 데이터셋이다. 이 단계에서는 빠르게 baseline 모델을 구축하는 것이 목적이었기 때문에, 데이터의 세부적인 라벨 품질이나 클래스 누락 여부에 대한 정밀 검토는 충분히 이루어지지 않았다. 이후 V2에서는 V1을 바탕으로 라벨링과 데이터 정제를 전반적으로 수행하여 보다 신뢰도 높은 데이터셋을 구축하였다. 발표자료의 데이터셋 변화 슬라이드에서 V1, V2가 baseline용 초기 데이터로 언급된 점은 이러한 흐름과 일치한다.

5.2 V3: 단일 객체 중심의 재가공 데이터셋

V3는 V2 데이터셋을 바탕으로 부족한 클래스를 보완하고, Stage 2 분류기를 위한 단일 객체 데이터를 새롭게 추출한 단계였다. 발표자료의 v3 stage2 데이터 가공 작업 슬라이드에서는 단일 객체 데이터 추출, 추출된 데이터의 분류 및 classid 부여, AI Hub 데이터에서의 추가 확보, 라벨링 작업 진행 등이 핵심 작업으로 제시된다. 이는 Stage 2 분류 성능 향상을 위해 데이터를 단순히 늘리는 것이 아니라, detector가 검출한 crop과 유사한 형태의 학습 데이터를 만드는 것이 중요하다고 판단했기 때문이다.

그러나 V3 기반 학습은 기대와 달리 전반적으로 낮은 성능을 보였다. 발표자료의 중기 모델 실험 결과에 따르면, v3 데이터로 학습한 모델은 public score가 0.6301~0.7727 수준으로 낮았고, 수직 반전, 회전 각도 조정, blur 확률 조절 등의 변화가 일부 영향을 주기는 했으나 전체적인 성능 문제를 해결하지는 못했다. 이는 stage1과 stage2에 사용된 데이터의 특성이 서로 완전히 일치하지 않았거나, 데이터 양과 품질, 그리고 augmentation 설계가 충분하지 않았기 때문으로 해석할 수 있다. 즉, V3는 Stage 2 분류를 위한 중요한 시도였으나, 이후 데이터 확장과 구조 재정립의 필요성을 드러낸 단계였다.

5.3 V4: AI Hub 기반 검증 데이터셋

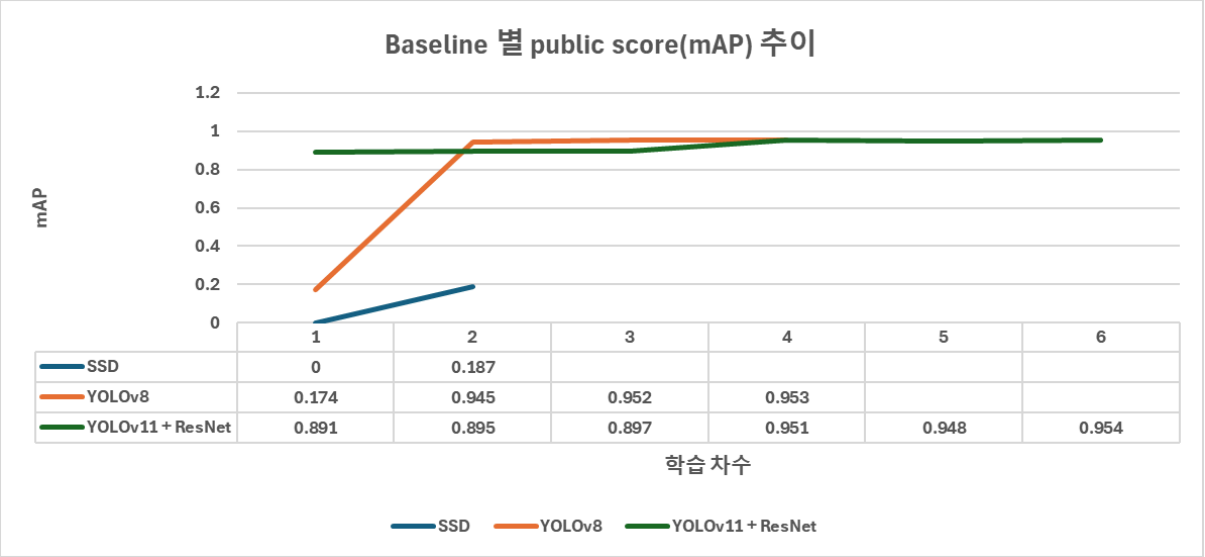
V4는 AI Hub에서 직접 활용 가능한 경구약제 단일 알약 기반 검증용 데이터셋으로 이해할 수 있다. 발표자료에서는 v4가 v3 대비 성능은 증가했지만 충분히 높은 기록은 아니었다고 정리되어 있으며, 동시에 v1+v3 조합에서 점차 더 높은 score가 기록되었다고 제시한다. 실제 표에서는 v4 실험이 최대 0.9655 수준을 보였고, v1+v3 조합은 0.98112까지 도달했다. 이 결과는 데이터셋의 출처 자체보다, 어떤 데이터들이 어떤 비율과 조건으로 결합되는지가 더 중요함을 시사한다.

5.4 V5와 V6: 확장 데이터 기반 최종 실험

V5는 AI Hub 전체 데이터 중 Kaggle train/test와 공통으로 존재하는 클래스들을 중심으로 재구성한 확장 데이터셋이었다. 발표자료의 v5 기본 설정, 실험 히스토리, 세부 실험 페이지들을 종합하면, 프로젝트 후반부의 핵심 성능 향상은 모두 V5 데이터를 중심으로 이루어졌다. 이후 V6는 AI Hub에 존재하는 전체 데이터셋을 모두 활용하는 방향으로 확장된 단계로 정리할 수 있다. 이처럼 데이터셋은 초기 baseline 데이터에서 시작하여, 단일 객체 중심 재가공, 검증 데이터셋 구축, 클래스 누락 보완, 저빈도 클래스 보강, 전체 데이터 확장이라는 흐름으로 발전하였다. 이러한 점은 본 프로젝트가 단순한 모델 튜닝보다, 데이터 중심의 성능 최적화 전략을 적극적으로 채택했음을 보여준다.

6. Baseline 모델 구축 및 비교

본 프로젝트는 최종 모델을 바로 선택하는 대신, 여러 모델 계열을 baseline 후보로 실험하고 비교하는 절차를 거쳤다. 발표자료의 baseline 비교 그래프에 따르면 SSD, YOLOv8, YOLOv11+ResNet 구조가 순차적으로 검토되었으며, 최고 성능은 각각 약 0.187, 0.953, 0.954 수준이었다. 이를 통해 최종적으로 YOLOv11s + ResNet50 구조가 baseline 모델로 선정되었다. 선정 근거로는 최고 mAP 달성, 역할 분리를 통한 성능 최적화, 정교한 feature 학습 가능성이 제시되었다.



[사진 1] v1을 활용한 모델별 score

6.1 SSD

SSD는 초기 baseline으로 도입되었으나, 본 문제에서 충분한 성능을 보여주지 못했다. 첫 실험에서는 mAP가 0.0에 머물러 실질적인 검출 성능을 확인하기 어려웠고, 이후 epoch 20, 기본 augmentation, class mapping 수정, brightness와 contrast 계열 augmentation을 적용한 후에도 결과는 0.18739 수준에 그쳤다. 발표자료는 이를 class mapping 오류와 구조적 성능 부족으로 해석하고 있다. 이는 SSD가 상대적으로 단순한 구조를 가지기 때문에, 알약처럼 fine-grained distinction이 필요한 객체 인식 문제에서는 feature 표현력이 충분하지 않았음을 의미한다.

구분	**(1)** 초기 **Baseline** 구축	**(2)** 오류 수정 및 증강기법 적용	**(3)** 추가 학습 중지
사용 모델	SSD	SSD	- 타 모델의 비해 성능 부족
결과 (mAP)	0.0	**0.18739**	
이전 학습 대비 변경점	- Epoch 20 - 기본 augmentation	- class mapping - bright, contrast 등 사용	
해석	- 성능 파악 불가 - class mapping 오류	- 성능 차이 미비 - 오류 개선	

6.2 YOLOv8

YOLOv8은 SSD에 비해 상당한 성능 향상을 보였다. YOLOv8n 기반 초기 실험은 0.17443으로 높지 않았으나, 모델 capacity를 YOLOv8m으로 확장하자 성능이 0.94475로 급상승하였다. 이후 epoch 증가와 augmentation 변경을 적용한 실험에서는 0.95195를 기록하였고, 마지막으로 모델을 YOLOv8s로 조정하여 0.953의 성능을 확보하였다. 발표자료는 이러한 변화를 모델의 feature 표현력 및 학습 안정화와 연결하여 해석하고 있다. YOLOv8은 baseline detector로서 충분히 경쟁력 있는 성능을 보였으나, 여전히 알약의 세밀한 문자 구분까지 하나의 모델에서 모두 처리하기에는 한계가 있었다.

구분	**(1)** 초기 **Baseline** 구축	**(2)** 모델 버전 변경	**(3)** 학습 전략 개선	**(4)** 모델 버전 변경
사용 모델	YOLOv8n	YOLOv8m	YOLOv8m	YOLOv8s
결과 (mAP)	0.17443	0.94475	0.95195	**0.953**
이전 학습 대비 변경점	- Epoch 20 - 기본 augmentation	- 모델 capacity 증가 (n → m)	- Epoch 증가 - augmentation 변경	- 모델 경량화 (m → s)
해석	- 성능 매우 낮음 - feature 표현력 부족	- 성능 급상승 - 모델 크기 영향이 매우 큼	- 학습 안정화 및 미세 성능 향상	- 성능 미세 향상 +효율 개선 - 최적 모델 도출

6.3 YOLOv11 + ResNet

가장 우수한 성능을 보인 구조는 detector와 classifier를 분리한 YOLOv11 + ResNet의 2-stage 구조였다. 발표자료에 따르면 YOLOv8n + ResNet18 조합으로 시작한 초기 구조는 0.89134의 mAP를 기록하였다. 이후 Stage 2 backbone을 ResNet34, ResNet50으로 변경하면서 성능은 0.89471, 0.89748로 점진적으로 향상되었다. 그 다음 Stage 1 detector를 YOLOv11n에서 YOLOv11s로 변경하면서 성능은 0.95142로 크게 상승하였고, YOLOv11m으로 확장했을 때는 오히려 0.94808로 소폭 감소하였다. 마지막으로 밝기, 상하 반전, 회전, 선명도 증강을 추가 적용하자 0.95426의 최고 성능을 기록하였다. 이 결과는 Stage 1 detector의 품질이 전체 성능에 매우 큰 영향을 미치며, 동시에 Stage 2에서는 충분한 capacity를 가진 backbone이 필요한 문제임을 보여준다.

구분		**(1)** 초기 **Baseline** 구축	**(2)** 모델 버전 변경	**(3)** 모델 버전 변경	**(4)** 모델 버전 변경	**(5)** 모델 버전 변경	**(6) augmentation** 적용
사용 모델	stage 1	YOLOv8n	YOLOv11n	YOLOv11n	YOLOv11s	YOLOv11m	YOLOv11s
	stage 2	ResNet18	ResNet34	ResNet50	ResNet50	ResNet50	ResNet50
결과 (mAP)		0.89134	0.89471	0.89748	0.95142	**0.94808**	**0.95426**
이전 학습 대비 변경점		- Epoch 20 - yolo 기본 augmentation 적용	ResNet 모델 변경 (18 → 34)	ResNet 모델 변경 (34 → 50)	YOLO 모델 변경 (n -> s)	YOLO 모델 변경 (s -> m)	밝기, 상하 반전, 회전, 선명도 증강 추가
해석		Baseline 구축 및 기본 성능 확보	Stage2 capacity 증가 → 소폭 성능 향상	Stage2 추가 확장 → 안정적 성능 개선	Stage1 capacity 증가 → 큰 성능 향상	Stage1 추가 확장 → 성능 소폭 변동	Augmentation 적용 → 최고 성능 달성

7. 데이터셋 및 증강 기반 성능 개선 실험

프로젝트 후반부의 핵심은 V5 데이터셋을 기반으로 한 체계적인 성능 개선 실험이었다. 발표자료의 v5 실험 히스토리 그래프에 따르면, 실험은 데이터 보강과 증강 기법을 반복적으로 조정하는 방식으로 전개되었으며, mAP는 0.96369에서 시작해 0.98569까지 지속적으로 향상되었다. 이 과정은 단순한 trial-and-error가 아니라, 오류 사례를 분석하고 원인을 구조적으로 해석한 뒤 그에 대응하는 실험을 설계하는 방식으로 이루어졌다.

7.1 누락 클래스 보강: No Class

[사진 2 누락된 데이터셋]

발표자료의 v5 No Class 실험에 따르면, Kaggle Train Images에는 포함되지 않은 클래스가 약 18개 존재하였다. 이에 대해 AI Hub에서 각 3~30개 수준의 데이터를 추가 확보하여 학습 데이터에 보강하였다. 이 실험은 총 281장의 이미지와 23분의 학습 시간으로 수행되었으며, mAP는 0.96369를 기록하였다. 이 결과는 특정 클래스가 train set에 존재하지 않을 경우, 모델은 구조적으로 그 클래스를 식별할 수 없다는 점을 보여준다. 따라서 클래스 coverage 확보는 본 문제에서 가장 기본적이면서도 중요한 데이터 전처리 단계였다.

7.2 앞·뒷면 불균형 문제: **No Back Or Front Print**

사진 3 앞뒷 사진

다음 실험에서는 특정 타겟의 예측 성능이 낮은 원인을 분석한 결과, 앞면과 뒷면의 식별 문자가 서로 다른 형태의 알약이었지만 뒷면 이미지가 Train Data에 존재하지 않음을 확인하였다. 이에 따라 데이터 부족이 성능 저하의 원인일 수 있다는 가설을 세우고, 문제 클래스뿐 아니라 약 6개 클래스에 대해 누락된 앞·뒷면 이미지를 추가하여 데이터 보강을 수행하였다. 그 결과 전체 mAP는 0.96451로 소폭 향상되었지만, 특정 객체에 대해서는 여전히 낮은 성능이 관찰되었다. 이 실험은 데이터 보강이 전반적 성능에는 기여할 수 있으나, 특정 문제 클래스의 경우 보다 정교한 접근이 필요함을 보여주었다.

7.3 문자 정보 강조: **Grayscale**

사진 4

발표자료에서는 모델이 식별 문자보다는 알약의 형태와 색상 정보를 더 중요하게 학습하고 있다고 판단하였고, 이를 바탕으로 문자 정보를 상대적으로 부각시키기 위해 Grayscale 증강을 적용하였다고 설명한다. 총 이미지 수 320장 조건에서 수행된 이 실험은 mAP 0.96634를 기록하였으며, 이전에 문제가 되었던 알약이 정상적으로 인식되는 사례를

확인할 수 있었다. 이는 알약 인식 문제에서 색상 자체가 도움이 되는 경우도 있지만, 특정 클래스들에서는 문자 가독성을 높이는 방향의 증강이 더 효과적일 수 있음을 보여준다.

7.4 Sharpen 적용과 한계: Grayscale & Sharpenorigin

Sharpen Grayscale Sharpen Grayscale

이후 문자 정보를 더욱 또렷하게 강조하기 위해 Sharpen 증강을 추가하였다. 직관적으로는 식별 문자의 윤곽이 선명해져 성능이 향상될 것으로 기대되었으나, 실제 결과는 예상과 달랐다. 발표자료에 따르면 총 이미지 수 320장 조건에서 mAP는 0.96329로 오히려 감소하였다. 다만 예측 결과를 분석해 보면 일부 객체에서는 Sharpen 증강이 성능 개선에 기여하는 사례도 존재하였다. 이 결과는 augmentation의 효과가 보편적으로 동일하지 않으며, 클래스의 특징에 따라 차별적으로 작용할 수 있음을 의미한다.

7.5 극저빈도 클래스 보강: Issue Class Data Add

특정 클래스 3743의 경우, 형태와 색상이 유사한 다른 알약과의 혼동이 반복적으로 발생하였다. 이에 대해 Grayscale, Sharpen, Sharpen+Grayscale의 조합을 모두 실험하였으나, 해당 객체를 정확히 식별하는 데에는 여전히 어려움이 있었다. 추가 분석 결과 이 클래스의 학습 데이터 수가 단 3개에 불과한 것으로 확인되었고, 이에 데이터를 추가 확보하여 재학습을 수행하였다. 그 결과 mAP는 0.96869까지 향상되었으며, 문제 객체가 이전보다 안정적으로 식별되는 경향을 확인하였다. 이 실험은 데이터 수가 극단적으로 적은 경우에는 augmentation보다 데이터 확보가 더 근본적인 해결책일 수 있음을 보여준다.

7.6 전체 저빈도 클래스 보강: All Issue Class Data Add

이전 실험 결과를 토대로, 이미지 수가 3~10개 수준에 불과한 클래스들은 단순한 augmentation만으로는 충분한 개선이 어렵다고 판단하였다. 이에 팀은 v3 단계에서 정리해 둔 알약 정보와 분포 자료를 참고하여, 해당 클래스들에 대한 데이터를 AI Hub에서 추가 확보하고 전체 분포를 보강하였다. 이후 기본 증강에 Grayscale을 추가한 상태로 학습을 수행한 결과, 총 이미지 수 2,312장 조건에서 mAP 0.97483을 기록하였다. 이는 저빈도 클래스 보강이 전체 성능 향상에 실질적인 영향을 미쳤음을 보여주는 결과였다.

참조: pill_dataset_정리

7.7 클래스별 맞춤형 증강: Class By Augmentation

구분	증강 항목	세부 설정	적용 확률
공통 증강	회전	0° 또는 180° 회전	30%
	회전	0° ~ 5° 랜덤 회전	30%
	색상 변환	Grayscale 적용	20%
	선명도 향상	ImageFilter.SHARPEN (약한 강도)	30%
Grayscale 주요 타겟	색상 변환	공통 증강 + Grayscale 추가 적용	40%
Sharpen 주요 타겟	선명도 향상	공통 증강 + UnsharpMask (radius=2, percent=150, threshold=3)	40%

표 4 클래스별 증강 현황

최종적으로 팀은 클래스별로 효과적인 augmentation이 다르다는 점에 주목하였다. 발표자료에 따르면, 공통 증강으로 0° 또는 180° 회전, 소각도 회전, 낮은 확률의 Grayscale, 약한 강도의 Sharpen을 적용하였고, 여기에 더해 Grayscale이 효과적이었던 객체들과 Sharpen이 효과적이었던 객체들을 별도의 리스트로 관리하여 클래스별 맞춤 증강이 이루어지도록 하였다. 이 실험은 총 이미지 수 2,312장 조건에서 mAP 0.98174를 기록하였다. 이는 전체 데이터셋에 동일한 augmentation을 일괄 적용하는 전략보다 클래스 특성을 고려한 선택적 augmentation이 더 우수한 결과를 낼 수 있음을 의미한다.

7.8 최종 최고 성능 실험

<table>
<tr><th colspan="3">구분</th><th colspan="2">설정</th></tr>
<tr><td rowspan="5">1 stage</td><td rowspan="5">bbox 검출</td><td>model</td><td colspan="2">YOLOv11s</td></tr>
<tr><td>optimizer</td><td colspan="2">AdamW</td></tr>
<tr><td>loss</td><td colspan="2">CrossEntropyLoss</td></tr>
<tr><td>augmentation</td><td colspan="2">auto</td></tr>
<tr><td>epochs</td><td colspan="2">20</td></tr>
<tr><td rowspan="7">2 stage</td><td rowspan="7">class 분류</td><td>model backbone</td><td colspan="2">ResNet50</td></tr>
<tr><td rowspan="2">augmentation</td><td>color jitter</td><td>random affine</td></tr>
<tr><td>gaussian blur</td><td>-</td></tr>
<tr><td>optimizer</td><td colspan="2">Adam</td></tr>
<tr><td>train images</td><td colspan="2">4719</td></tr>
<tr><td>epochs</td><td colspan="2">20</td></tr>
<tr><td colspan="3">Public Score(mAP)</td><td colspan="2">0.98569</td></tr>
</table>

발표자료의 v5 Base Augmentation By 4719 Images 페이지에 따르면, Stage 1에는 YOLOv11s, Stage 2에는 ResNet50 backbone을 사용하였고, Stage 2 증강으로 color jitter, random affine, gaussian blur, grayscale을 적용한 실험에서 최종적으로 Public Score(mAP) 0.98569를 달성하였다. 이 실험은 4,719장의 train images를 활용한 최종 설정으로, 본 프로젝트 전체 실험 중 가장 우수한 성능을 보인 결과였다. 따라서 본 보고서에서는 이를 최종 모델의 대표 성능으로 제시한다.

8. 최종 모델 선정 및 성능 분석

최종적으로 선정된 모델은 YOLOv11s + ResNet50 기반의 2-stage 구조였다. Stage 1의 YOLOv11s는 알약의 위치를 검출하는 역할을 담당하고, Stage 2의 ResNet50은 각 bounding box crop에 대해 세부 클래스를 분류하는 역할을 수행한다. 이러한 구조는 검출과 분류를 하나의 네트워크에 모두 맡기는 방식보다 각 단계가 문제의 서로 다른 난점에 집중할 수 있도록 해준다. 특히 알약 인식 문제에서는 위치 탐색과 세밀한 문자·형태 식별이 동시에 필요하기 때문에, 2-stage 구조가 구조적으로 더 적합했다. 발표자료의 baseline 선정 슬라이드에서도 이 점이 “역할 분리로 성능 최적화”라는 표현으로 정리되어 있다.

최종 모델이 높은 성능을 달성할 수 있었던 이유는 크게 세 가지로 해석할 수 있다. 첫째, detector와 classifier의 역할을 분리하여 각 단계의 최적화를 가능하게 하였다. 둘째, Stage 2에 ResNet50을 사용함으로써 알약 표면의 미세한 식별 정보를 더 풍부하게 표현할 수 있었다. 셋째, 데이터셋 자체를 V1~V6로 발전시키며 문제 클래스를 지속적으로 보강했고, 클래스별 augmentation 전략을 도입함으로써 단순한 모델 튜닝 이상의 성능 향상을 이끌어냈다. 즉, 최종 성능은 모델 구조, 데이터 품질, 실험 전략의 결합 결과라고 볼 수 있다.

9. 모델의 한계점

발표자료는 최종 모델이 높은 mAP를 기록했음에도 불구하고, 여전히 두 가지 구조적 한계를 가진다고 명시한다. 첫 번째는 **Closed-Set Classification** 문제이다. 현재의 Stage 2는 classifier head 기반 구조이므로, 학습 시 정의된 클래스 집합 밖의 새로운 알약이 입력되더라도 이를 별도의 unknown으로 처리하지 못하고 기존 클래스 중 하나로 강제 분류할 가능성이 높다. 이는 실제 서비스 환경에서 매우 중요한 문제이다. 실제 약품 환경에서는 신제품이나 미등록 약물이 등장할 수 있기 때문에, closed-set 구조만으로는 안전성과 확장성을 완전히 확보할 수 없다.

두 번째 한계는 신규 알약 추가 시 재학습이 필요한 문제이다. 현재 구조에서는 새로운 클래스를 추가하려면 분류기의 최종 출력층 구조가 바뀌어야 하므로, 모델을 다시 학습해야 하는 부담이 존재한다. 이는 서비스 유지보수 측면에서 비용이 크고, 클래스 수가 계속 증가하는 실제 상황에서는 비효율적일 수 있다. 따라서 본 프로젝트는 높은 정량 성능을 확보했음에도 불구하고, 실제 상용화를 위해서는 보다 유연한 분류 구조로의 발전이 필요하다는 점을 보여준다.

10. 향후 개선 방향

구분			기존 **(Classifier Head)**	변경 **(Embedding Head)**
1 stage	bbox 검출	model	YOLOv11s	
		optimizer	AdamW	
		loss	CrossEntropyLoss	
		augmentation	auto	
		epochs	20	
2 stage	class 분류	model backbone	ResNet50	
		model head	FC layer + Softmax	Embedding Vector
		loss	CrossEntopyLoss	SupConLoss
		optimizer	Adam	
		augmentation	color jitter, random affine, gaussian blur, vertical flip	
		epochs	20	
Public Score(mAP)			**0.98569**	**0.97878**

이러한 한계를 해결하기 위해 Stage 2에 **Embedding Head**를 적용하는 방안을 제안한다. 기존 구조에서는 ResNet50 backbone 뒤에 FC layer와 Softmax를 두어 고정 클래스 분류를 수행하였으나, embedding head 구조에서는 각 알약 이미지를 임베딩 벡터로 매핑하고, 벡터 간 거리나 유사도를 기반으로 클래스를 판별할 수 있다. 이 방식의 가장 큰 장점은 미등록 클래스를 unknown으로 탐지할 가능성이 생긴다는 점이며, 새로운 클래스가 추가되더라도 전체 분류기를 재설계하는 부담이 상대적으로 줄어든다는 점이다. 발표자료는 이를 향후 해결 방안으로 명시하고 있다.

다만, embedding head를 실제로 적용한 발표자료의 실험 결과를 보면 public mAP는 0.97878로, classifier head 기반 최종 모델의 0.98569보다 다소 낮았다. 이는 embedding 구조가 확장성과 개방 집합 문제 해결에는 유리할 수 있지만, 현재의 fixed-class benchmark 환경에서는 여전히 softmax classifier가 더 높은 정량 성능을 낼 수 있음을 시사한다. 따라서 향후에는 단순 mAP뿐 아니라 unknown detection, class expansion cost, embedding space stability와 같은 새로운 평가 지표를 함께 고려하는 연구가 필요하다.

11. 결론

본 프로젝트는 이미지 기반 알약 인식 문제를 대상으로, 객체 검출과 세부 분류를 결합한 2-stage 헬스케어 AI 시스템을 개발하고 이를 체계적으로 고도화한 사례이다. 초기에는 Kaggle 데이터와 baseline 모델 실험을 통해 출발하였으나, 프로젝트가 진행됨에 따라 데이터셋 정제, 누락 클래스 보강, 앞·뒷면 데이터 불균형 해소, Grayscale과 Sharpen 증강 실험, 저빈도 클래스 데이터 추가, 클래스별 맞춤형 augmentation 설계 등 문제 중심의 개선 전략을 지속적으로 축적하였다. 이러한 접근은 단순한 모델 변경보다 데이터와 오류 분석을 중심으로 성능을 향상시키는 데 효과적이었다.

최종적으로 YOLOv11s + ResNet50 구조와 4,719장 규모의 학습 이미지, 적절한 augmentation 조합을 통해 Public Score(mAP) 0.98569를 달성하였다. 이는 본 프로젝트가 높은 정량 성능을 확보했음을 보여주는 동시에, 실제 서비스 적용 가능성 측면에서도 의미 있는 결과라고 평가할 수 있다. 동시에 본 보고서는 현재 구조가 closed-set classification과 신규 클래스 추가 문제라는 한계를 가진다는 점을 분석하였고, 향후 embedding head 기반 구조로의 확장이 이를 해결할 수 있는 유망한 방향임을 제시하였다. 따라서 본 프로젝트는 단순한 성능 최적화 결과를 넘어, 실제 헬스케어 서비스로 발전 가능한 알약 인식 시스템의 기초 설계와 개선 전략을 제시한 사례로 의의를 가진다.